

Atividade 3 – Types e Interfaces em TypeScript

Técnicas de Programação II - Alicia Silva Dias

Exercício 1 - Interseção de Tipos em TypeScript

Para satisfazer a interseção $X \& Y \& \{d: \text{string}\}$, o objeto passado como argumento precisa conter **todas** as propriedades exigidas por X (a e b), por Y (c), além da propriedade explícita d.

```
JS exercicio1 > ...  
1      um({  
2          a: 10,  
3          b: 20,  
4          c: 30,  
5          d: "Exemplo de String"  
6      });
```

Exercício 2 - União e Interseção de Tipos em TypeScript

Primeira chamada ({a: 1, b: 2, d: "oi"}) - CORRETA: Fornece as propriedades exigidas pelo tipo X (a e b) e cumpre a exigência da propriedade d.

Segunda chamada ({c: 1, d: "oi"}) - CORRETA: Fornece a propriedade exigida pelo tipo Y (c) e cumpre a exigência da propriedade d.

Terceira chamada ({a: 1, d: "oi"}) - INCORRETA: Embora possua a propriedade d, ela falha em satisfazer o tipo Z. Para ser considerado do tipo X, faltou passar a propriedade obrigatória b.

Quarta chamada ({d: "oi"}) - INCORRETA: O objeto possui apenas a propriedade d. Ela não atende nem aos requisitos de X (falta a e b) e nem aos requisitos de Y (falta c), tornando a união inválida.

Exercício 3 - Conversão de Types para Interfaces

As estruturas X e Y podem ser facilmente convertidas para interfaces. No entanto, **interfaces não conseguem representar uniões (|) diretamente**. A herança (extends) de uma interface só

funciona para interseções (&). Portanto, a alternativa correta é manter a definição de união Z usando type, mas apontando para as novas interfaces.

```
TS exercicio3 >  dois
1  // Conversão direta de X e Y para interfaces
2  interface X {
3      a: number;
4      b: number;
5  }
6
7  interface Y {
8      c: number;
9  }
10
11 // Justificativa/Alternativa: Interfaces não suportam uniões diretas (X | Y).
12 // Portanto, Z precisa continuar sendo um alias de tipo (type) que combina as duas interfaces.
13 type Z = X | Y;
14
15 function dois(w: Z & { d: string }) {
16     console.log(w);
17 }
```

Exercício 4 - Refatoração de Types para Interfaces

```
TS exercicio4 >  um
1  interface X {
2      a: number;
3      b: number;
4  }
5
6  interface Y {
7      c: number;
8  }
9
10 // Usando herança com extends para fazer o papel da interseção (&)
11 interface ParametroFuncao extends X, Y {
12     d: string;
13 }
14
15 function um(w: ParametroFuncao) {
16     console.log(w);
17 }
```

Exercício 5 - Diferenças entre type e interface no TypeScript

Duas diferenças:

- **Declaration Merging (Mesclagem de Declarações):** Se declarar duas interfaces com o mesmo nome no mesmo escopo, o TypeScript junta as duas em uma só. Com type, o TypeScript gera um erro de identificador duplicado.
- **Capacidade de União/Primitivos:** Um type consegue mapear tipos primitivos diretos, uniões (|) e tuplas, enquanto a interface serve estritamente para descrever a estrutura de objetos e funções.

Quando usar Interface: É recomendada na modelagem de arquiteturas orientadas a objetos clássicas (POO), na criação de contratos públicos para APIs ou bibliotecas (devido à mesclagem extensível) e na definição de componentes visuais em frameworks (como Props em React).

Quando usar Type: É adequado quando se precisa usar recursos avançados do sistema de tipos do TypeScript, tais como uniões de tipos literais (ex: status fixos), interseções complexas, tipos utilitários (Utility Types), mapeamentos ou manipulação de tipos primitivos.

Exercício 6 - Alias Complexos com type no TypeScript

Tipo Status: Representa um tipo literal de união (*Union Type*). Significa que qualquer variável associada a esse tipo só poderá armazenar e aceitar estritamente uma das três strings declaradas: "ativo", "inativo" ou "bloqueado".

Tipo UsuarioSistema: Representa uma interseção (*Intersection Type*) que combina a estrutura de Usuario (que contém id e nome) com um novo objeto que adiciona obrigatoriamente a propriedade status (do tipo Status) e opcionalmente a propriedade ultimoAcesso.

Por que não pode ser substituída por Interface: Porque interfaces servem apenas para ditar a estrutura/forma de objetos e classes. Elas não têm suporte sintático para expressar tipos primitivos puros ou coleções alternativas através de operadores de união (|).